```
Start coding or generate with AI.
```

```
import pandas as pd

# Load the dataset, assuming it's tab-separated and doesn't have a header initially
df = pd.read_csv('/content/SMSSpamCollection', sep='\t', header=None)

# Display the first 5 rows to inspect the data structure
display(df.head())
```

	0	1
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

```
import nltk
nltk.download('punkt')
nltk.download('stopwords')
print("NLTK and its required resources ('punkt', 'stopwords') have been imported and downloaded.")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
NLTK and its required resources ('punkt', 'stopwords') have been imported and downloaded.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```
print(f"Shape of the DataFrame: {df.shape}")
```

```
Shape of the DataFrame: (5572, 2)
```

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   0       5572 non-null   object
 1   1       5572 non-null   object
dtypes: object(2)
memory usage: 87.2+ KB
None
```

```
# Rename columns for better readability
df.columns = ['label', 'message']

# Display the first few rows with new column names
display(df.head())
```

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Reasoning: The first step in text cleaning is to convert all text in the 'message' column to lowercase to ensure uniformity, and store it in a new column called 'cleaned_message'.

```
df['cleaned_message'] = df['message'].str.lower()
display(df.head())
```

	label	message	cleaned_message
0	ham	Go until jurong point, crazy.. Available only ...	go until jurong point, crazy.. available only ...
1	ham	Ok lar... Joking wif u oni...	ok lar... joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	free entry in 2 a wkly comp to win fa cup fina...
3	ham	U dun say so early hor... U c already then say...	u dun say so early hor... u c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...	nah i don't think he goes to usf, he lives aro...

```
import string
def remove_punctuation(text):
    return text.translate(str.maketrans('', '', string.punctuation))
df['cleaned_message'] = df['cleaned_message'].apply(remove_punctuation)
display(df.head())
```

	label	message	cleaned_message
0	ham	Go until jurong point, crazy.. Available only ...	go until jurong point crazy available only in ...
1	ham	Ok lar... Joking wif u oni...	ok lar joking wif u oni
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	free entry in 2 a wkly comp to win fa cup fina...
3	ham	U dun say so early hor... U c already then say...	u dun say so early hor u c already then say
4	ham	Nah I don't think he goes to usf, he lives aro...	nah i dont think he goes to usf he lives aroun...

```
df['cleaned_message'] = df['cleaned_message'].str.replace(r'\d+', '', regex=True)
display(df.head())
```

	label	message	cleaned_message
0	ham	Go until jurong point, crazy.. Available only ...	go until jurong point crazy available only in ...
1	ham	Ok lar... Joking wif u oni...	ok lar joking wif u oni
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	free entry in a wkly comp to win fa cup final...
3	ham	U dun say so early hor... U c already then say...	u dun say so early hor u c already then say
4	ham	Nah I don't think he goes to usf, he lives aro...	nah i dont think he goes to usf he lives aroun...

```
df['cleaned_message'] = df['cleaned_message'].str.replace(r'[^a-zA-Z\s]', '', regex=True)
display(df.head())
```

	label	message	cleaned_message
0	ham	Go until jurong point, crazy.. Available only ...	go until jurong point crazy available only in ...
1	ham	Ok lar... Joking wif u oni...	ok lar joking wif u oni
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	free entry in a wkly comp to win fa cup final...
3	ham	U dun say so early hor... U c already then say...	u dun say so early hor u c already then say
4	ham	Nah I don't think he goes to usf, he lives aro...	nah i dont think he goes to usf he lives aroun...

Tokenization

Subtask:

Tokenize the cleaned messages into individual words using NLTK's word_tokenize.

Reasoning: To tokenize the cleaned messages, I will import `word_tokenize` from `nltk.tokenize`, apply it to the `cleaned_message` column to create a new `tokenized_message` column, and then display the DataFrame head to verify the changes.

```
from nltk.tokenize import word_tokenize
df['tokenized_message'] = df['cleaned_message'].apply(word_tokenize)
display(df.head())

# The previous execution failed because 'punkt_tab' resource was missing.
# This step is intended to be re-executed after downloading 'punkt_tab'.
```

	label	message	cleaned_message	tokenized_message	filtered_message	lemmatized_message	message_length	processed_messa
0	ham	Go until jurong point, crazy.. Available only ...	go until jurong point crazy available only in ...	[go, until, jurong, point, crazy, available, o...	[go, jurong, point, crazy, available, bugis, n...	[go, jurong, point, crazy, available, bugis, n...	16	go jurong point cra available bugi grea
1	ham	Ok lar... Joking wif u oni...	ok lar joking wif u oni	[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]	6	ok lar joking wif u
2	spam	Free entry in 2 a wkly comp to win FA Cup	free entry in a wkly comp to win fa cup final...	[free, entry, in, a, wkly, comp, to, win, fa, ...	[free, entry, wkly, comp, win, fa, cup, final,...	[free, entry, wkly, comp, win, fa, cup, final,...	20	free entry wkly co win fa cup final t s

```
import nltk
nltk.download('punkt_tab')
print("NLTK resource 'punkt_tab' downloaded.")
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
NLTK resource 'punkt_tab' downloaded.
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

Reasoning: Since the `punkt_tab` resource has now been successfully downloaded, I will re-run the tokenization code to create the `tokenized_message` column, which should now execute without error.

```
from nltk.tokenize import word_tokenize
df['tokenized_message'] = df['cleaned_message'].apply(word_tokenize)
display(df.head())
```

	label	message	cleaned_message	tokenized_message
0	ham	Go until jurong point, crazy.. Available only ...	go until jurong point crazy available only in ...	[go, until, jurong, point, crazy, available, o...
1	ham	Ok lar... Joking wif u oni...	ok lar joking wif u oni	[ok, lar, joking, wif, u, oni]
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	free entry in a wkly comp to win fa cup final...	[free, entry, in, a, wkly, comp, to, win, fa, ...
3	ham	U dun say so early hor... U c already then say...	u dun say so early hor u c already then say	[u, dun, say, so, early, hor, u, c, already, t...

```
from nltk.corpus import stopwords

# Create a set of English stopwords
stop_words = set(stopwords.words('english'))

# Define a function to remove stopwords
def remove_stopwords(tokens):
    return [word for word in tokens if word not in stop_words]

# Apply the function to the 'tokenized_message' column
df['filtered_message'] = df['tokenized_message'].apply(remove_stopwords)

# Display the first few rows of the DataFrame to inspect the new column
display(df.head())
```

	label	message	cleaned_message	tokenized_message	filtered_message
0	ham	Go until jurong point, crazy.. Available only ...	go until jurong point crazy available only in ...	[go, until, jurong, point, crazy, available, o...	[go, jurong, point, crazy, available, bugis, n...
1	ham	Ok lar... Joking wif u oni...	ok lar joking wif u oni	[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	free entry in a wkly comp to win fa cup final...	[free, entry, in, a, wkly, comp, to, win, fa, ...	[free, entry, wkly, comp, win, fa, cup, final,...
3	ham	U dun say so early hor... U c already then say...	u dun say so early hor u c already then say	[u, dun, say, so, early, hor, u, c, already, t...	[u, dun, say, early, hor, u, c, already, say]

```
import nltk
nltk.download('wordnet')
print("NLTK resource 'wordnet' downloaded.")
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
NLTK resource 'wordnet' downloaded.
```

```
from nltk.stem import WordNetLemmatizer

# Instantiate the WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

# Define a function to lemmatize tokens
def lemmatize_tokens(tokens):
    return [lemmatizer.lemmatize(word) for word in tokens]

# Apply the lemmatization function to the 'filtered_message' column
df['lemmatized_message'] = df['filtered_message'].apply(lemmatize_tokens)

# Display the first few rows of the DataFrame to inspect the new column
display(df.head())
```

	label	message	cleaned_message	tokenized_message	filtered_message	lemmatized_message
0	ham	Go until jurong point, crazy.. Available only ...	go until jurong point crazy available only in ...	[go, until, jurong, point, crazy, available, o...	[go, jurong, point, crazy, available, bugis, n...	[go, jurong, point, crazy, available, bugis, n...
1	ham	Ok lar... Joking wif u oni...	ok lar joking wif u oni	[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	free entry in a wkly comp to win fa cup final...	[free, entry, in, a, wkly, comp, to, win, fa, ...	[free, entry, wkly, comp, win, fa, cup, final,...	[free, entry, wkly, comp, win, fa, cup, final,...
3	ham	U dun say so early hor... U c already then say...	u dun say so early hor u c already then say	[u, dun, say, so, early, hor, u, c, already, t...	[u, dun, say, early, hor, u, c, already, say]	[u, dun, say, early, hor, u, c, already, say]

```
display(df[['message', 'cleaned_message', 'tokenized_message', 'filtered_message', 'lemmatized_message']].head())
```

	message	cleaned_message	tokenized_message	filtered_message	lemmatized_message
0	Go until jurong point, crazy.. Available only ...	go until jurong point crazy available only in ...	[go, until, jurong, point, crazy, available, o...	[go, jurong, point, crazy, available, bugis, n...	[go, jurong, point, crazy, available, bugis, n...
1	Ok lar... Joking wif u oni...	ok lar joking wif u oni	[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]
2	Free entry in 2 a wkly comp to win FA Cup fina...	free entry in a wkly comp to win fa cup final...	[free, entry, in, a, wkly, comp, to, win, fa, ...	[free, entry, wkly, comp, win, fa, cup, final,...	[free, entry, wkly, comp, win, fa, cup, final,...
3	U dun say so early hor... U c already then say...	u dun say so early hor u c already then say	[u, dun, say, so, early, hor, u, c, already, t...	[u, dun, say, early, hor, u, c, already, say]	[u, dun, say, early, hor, u, c, already, say]

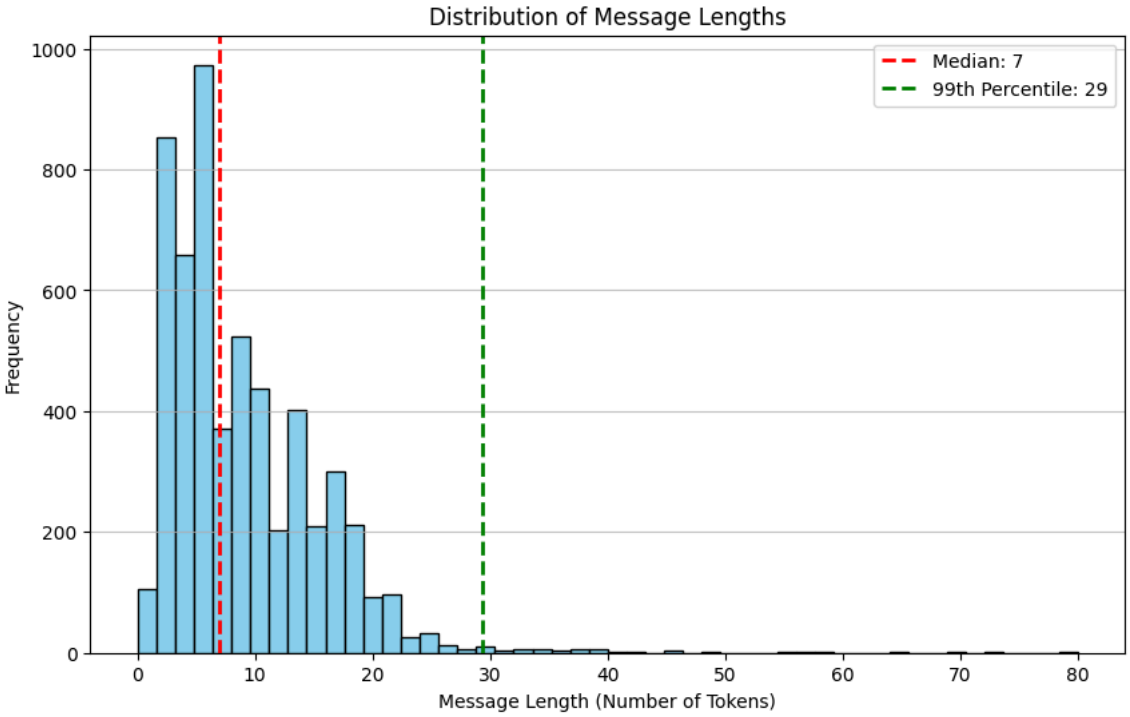
```
df['message_length'] = df['lemmatized_message'].apply(len)
display(df.head())
```

	label	message	cleaned_message	tokenized_message	filtered_message	lemmatized_message	message_length
0	ham	Go until jurong point, crazy.. Available only ...	go until jurong point crazy available only in ...	[go, until, jurong, point, crazy, available, o...	[go, jurong, point, crazy, available, bugis, n...	[go, jurong, point, crazy, available, bugis, n...	16
1	ham	Ok lar... Joking wif u oni...	ok lar joking wif u oni	[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]	6
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	free entry in a wkly comp to win fa cup final...	[free, entry, in, a, wkly, comp, to, win, fa, ...	[free, entry, wkly, comp, win, fa, cup, final,...	[free, entry, wkly, comp, win, fa, cup, final,...	20

```
import matplotlib.pyplot as plt

# Print descriptive statistics of the message_length column
print("Descriptive statistics for message length:")
print(df['message_length'].describe())

# Calculate median and 99th percentile for vertical lines
median_length = df['message_length'].median()
p99_length = df['message_length'].quantile(0.99)

# Create a histogram
plt.figure(figsize=(10, 6))
plt.hist(df['message_length'], bins=50, color='skyblue', edgecolor='black')
plt.title('Distribution of Message Lengths')
plt.xlabel('Message Length (Number of Tokens)')
plt.ylabel('Frequency')
```

```
# Add vertical lines for median and 99th percentile
plt.axvline(median_length, color='red', linestyle='dashed', linewidth=2, label=f'Median: {median_length:.0f}')
plt.axvline(p99_length, color='green', linestyle='dashed', linewidth=2, label=f'99th Percentile: {p99_length:.0f}')

plt.legend()
plt.grid(axis='y', alpha=0.75)
plt.show()
```

```
Descriptive statistics for message length:
count    5572.000000
mean        8.954415
std         6.506537
min         0.000000
25%         4.000000
50%         7.000000
75%        13.000000
max        80.000000
Name: message_length, dtype: float64
```




```
all_lemmas = sum(df['lemmatized_message'].tolist(), [])
print(f"Total number of lemmatized words: {len(all_lemmas)}")
print(f"First 10 lemmatized words: {all_lemmas[:10]}")
```

```
Total number of lemmatized words: 49894
First 10 lemmatized words: ['go', 'jurong', 'point', 'crazy', 'available', 'bugis', 'n', 'great', 'world', 'la']
```

```
from collections import Counter

# Count the frequency of each word
word_counts = Counter(all_lemmas)

# Print the total number of unique words
print(f"Total number of unique words: {len(word_counts)}")

# Display the 20 most common words and their frequencies
print("\n20 Most Common Words:")
for word, count in word_counts.most_common(20):
    print(f"{word}: {count}")
```

```
Total number of unique words: 7968

20 Most Common Words:
u: 1219
call: 605
im: 476
get: 401
ur: 390
go: 311
dont: 298
ok: 278
free: 278
ltgt: 276
know: 267
day: 256
got: 252
```

```
come: 250
like: 247
ill: 243
good: 237
time: 233
text: 215
want: 214
```

```
word_to_idx = {'<unk>': 0}  # Reserve index 0 for unknown words
idx = 1

for word, count in word_counts.items():
    if word not in word_to_idx:
        word_to_idx[word] = idx
        idx += 1

# Print the size of the vocabulary
print(f"Vocabulary size: {len(word_to_idx)}")

# Display the first 20 words and their indices to verify
print("\nFirst 20 words in vocabulary with their indices:")
for word, index in list(word_to_idx.items())[:20]:
    print(f"{word}: {index}")
```

```
Vocabulary size: 7969

First 20 words in vocabulary with their indices:
<unk>: 0
go: 1
jurong: 2
point: 3
crazy: 4
available: 5
bugis: 6
n: 7
great: 8
world: 9
la: 10
e: 11
buffet: 12
cine: 13
got: 14
amore: 15
wat: 16
ok: 17
lar: 18
joking: 19
```

```
!
!wget http://nlp.stanford.edu/data/glove.6B.zip
!unzip glove.6B.zip
print("GloVe embeddings downloaded and unzipped.")
```

```
--2026-02-19 03:59:05--  http://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2026-02-19 03:59:06--  https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-02-19 03:59:06--  https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: ‘glove.6B.zip’

glove.6B.zip        100%[===================>] 822.24M  5.05MB/s    in 2m 42s

2026-02-19 04:01:49 (5.07 MB/s) - ‘glove.6B.zip’ saved [862182613/862182613]

Archive:  glove.6B.zip
  inflating: glove.6B.50d.txt
  inflating: glove.6B.100d.txt
  inflating: glove.6B.200d.txt
  inflating: glove.6B.300d.txt
GloVe embeddings downloaded and unzipped.
```

```
import numpy as np

glove_embeddings = {}
file_path = '/content/glove.6B.100d.txt'

with open(file_path, 'r', encoding='utf8') as f:
```

```
    for line in f:
        parts = line.split()
        word = parts[0]
        vector = np.array(parts[1:], dtype=np.float32)
        glove_embeddings[word] = vector

print(f"Loaded {len(glove_embeddings)} GloVe embeddings.")
print(f"Embedding dimension: {len(glove_embeddings['the'])} (e.g., for word 'the')")
```

```
Loaded 400000 GloVe embeddings.
Embedding dimension: 100 (e.g., for word 'the')
```

```
vocabulary_size = len(word_to_idx)
embedding_dimension = 100

embedding_matrix = np.zeros((vocabulary_size, embedding_dimension))

print(f"Embedding matrix initialized with shape: {embedding_matrix.shape}")
```

```
Embedding matrix initialized with shape: (7969, 100)
```

```
for word, i in word_to_idx.items():
    if word in glove_embeddings:
        embedding_matrix[i] = glove_embeddings[word]

print(f"Embedding matrix populated. Shape: {embedding_matrix.shape}")

# Verify a few embedded words (e.g., 'go', 'jurong', 'crazy')
print(f"\nEmbedding for 'go' (first 5 values): {embedding_matrix[word_to_idx['go']][:5]}")
print(f"Embedding for 'jurong' (first 5 values): {embedding_matrix[word_to_idx['jurong']][:5]}")
print(f"Embedding for 'crazy' (first 5 values): {embedding_matrix[word_to_idx['crazy']][:5]}")
```

```
Embedding matrix populated. Shape: (7969, 100)

Embedding for 'go' (first 5 values): [-0.078894    0.46160001  0.57779002 -0.71636999 -0.13121   ]
Embedding for 'jurong' (first 5 values): [-0.1955      0.37338001  0.48943999  0.38391    -0.22717001]
Embedding for 'crazy' (first 5 values): [ 0.2721     -0.074994    0.91788    -0.70161998 -0.51916999]
```

```
df['processed_message'] = df['lemmatized_message'].apply(lambda x: ' '.join(x))
display(df.head())
```

	label	message	cleaned_message	tokenized_message	filtered_message	lemmatized_message	message_length	processed_messa
0	ham	Go until jurong point, crazy.. Available only ...	go until jurong point crazy available only in ...	[go, until, jurong, point, crazy, available, o...	[go, jurong, point, crazy, available, bugis, n...	[go, jurong, point, crazy, available, bugis, n...	16	go jurong point cra available bugi great
1	ham	Ok lar... Joking wif u oni...	ok lar joking wif u oni	[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]	6	ok lar joking wif u
2	spam	Free entry in 2 a wkly comp to win FA Cup	free entry in a wkly comp to win fa cup final...	[free, entry, in, a, wkly, comp, to, win, fa, ...	[free, entry, wkly, comp, win, fa, cup, final,...	[free, entry, wkly, comp, win, fa, cup, final,...	20	free entry wkly co win fa cup final t s

```
X = df['processed_message']
y = df['label']

print(f"Shape of X: {X.shape}")
print(f"Shape of y: {y.shape}")
print("First 5 elements of X:")
print(X.head())
print("First 5 elements of y:")
print(y.head())
```

```
Shape of X: (5572,)
Shape of y: (5572,)
First 5 elements of X:
0    go jurong point crazy available bugis n great ...
1                           ok lar joking wif u oni
2    free entry wkly comp win fa cup final tkts st ...
3                  u dun say early hor u c already say
4             nah dont think go usf life around though
Name: processed_message, dtype: object
First 5 elements of y:
0     ham
```

```
1     ham
2    spam
3     ham
4     ham
Name: label, dtype: object
```

```
from sklearn.preprocessing import LabelEncoder

# Instantiate LabelEncoder
le = LabelEncoder()

# Fit and transform the 'y' (label) Series
y = le.fit_transform(y)

# Display the first 5 elements of the encoded y
print("First 5 elements of encoded y:")
print(y[:5])

# Print the unique values in the encoded y and their counts
print("\nUnique values in encoded y and their counts:")
unique_values, counts = np.unique(y, return_counts=True)
for value, count in zip(unique_values, counts):
    print(f"{le.inverse_transform([value])[0]} ({value}): {count}")
```

```
First 5 elements of encoded y:
[0 0 1 0 0]

Unique values in encoded y and their counts:
ham (0): 4825
spam (1): 747
```

```
from sklearn.model_selection import train_test_split

print("train_test_split imported successfully.")
```

```
train_test_split imported successfully.
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

# Verify class distribution in training set
unique_train, counts_train = np.unique(y_train, return_counts=True)
print("\nClass distribution in y_train:")
for val, count in zip(unique_train, counts_train):
    print(f"Label {val}: {count} samples ({count/len(y_train)*100:.2f}%)")

# Verify class distribution in testing set
unique_test, counts_test = np.unique(y_test, return_counts=True)
print("\nClass distribution in y_test:")
for val, count in zip(unique_test, counts_test):
    print(f"Label {val}: {count} samples ({count/len(y_test)*100:.2f}%)")
```

```
Shape of X_train: (4457,)
Shape of X_test: (1115,)
Shape of y_train: (4457,)
Shape of y_test: (1115,)

Class distribution in y_train:
Label 0: 3859 samples (86.58%)
Label 1: 598 samples (13.42%)

Class distribution in y_test:
Label 0: 966 samples (86.64%)
Label 1: 149 samples (13.36%)
```

```
def text_to_sequences(text_list):
    """Converts a list of words (tokenized message) to a list of word indices."""
    return [word_to_idx.get(word, word_to_idx['<unk>']) for word in text_list]

# Apply the function to X_train and X_test to get numerical sequences
X_train_sequences = X_train.apply(lambda x: text_to_sequences(x.split()))
X_test_sequences = X_test.apply(lambda x: text_to_sequences(x.split()))

print("Converted text data to sequences of word indices.")
print(f"First sequence in X_train_sequences: {X_train_sequences.iloc[0][:10]}")
print(f"First sequence in X_test_sequences: {X_test_sequences.iloc[0][:10]}")

from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
# Convert p99_length to an integer for maxlen
max_sequence_length = int(p99_length)

# Pad sequences to a uniform length
X_train_padded = pad_sequences(X_train_sequences, maxlen=max_sequence_length, padding='post')
X_test_padded = pad_sequences(X_test_sequences, maxlen=max_sequence_length, padding='post')

print(f"\nPadding applied. Max sequence length: {max_sequence_length}")
print(f"Shape of X_train_padded: {X_train_padded.shape}")
print(f"Shape of X_test_padded: {X_test_padded.shape}")
```

```
Converted text data to sequences of word indices.
First sequence in X_train_sequences: [924, 538]
First sequence in X_test_sequences: [321, 566, 213, 223, 5459, 5460]

Padding applied. Max sequence length: 29
Shape of X_train_padded: (4457, 29)
Shape of X_test_padded: (1115, 29)
```

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Embedding, Conv1D, GlobalMaxPooling1D, Dense, concatenate

print("Keras modules and layers imported successfully.")
```

```
Keras modules and layers imported successfully.
```

```
input_layer = Input(shape=(max_sequence_length,))

embedding_layer = Embedding(
    input_dim=vocabulary_size,
    output_dim=embedding_dimension,
    weights=[embedding_matrix],
    input_length=max_sequence_length,
    trainable=False
)(input_layer)

# Define different filter sizes for Conv1D layers
filter_sizes = [3, 4, 5]
conv_blocks = []

for fsize in filter_sizes:
    conv = Conv1D(filters=128, kernel_size=fsize, activation='relu')(embedding_layer)
    pool = GlobalMaxPooling1D()(conv)
    conv_blocks.append(pool)

# Concatenate the outputs of the different convolutional blocks
if len(conv_blocks) > 1:
    merged = concatenate(conv_blocks)
else:
    merged = conv_blocks[0]

dense_layer_1 = Dense(128, activation='relu')(merged)
output_layer = Dense(1, activation='sigmoid')(dense_layer_1)

model = Model(inputs=input_layer, outputs=output_layer)

print("CNN model architecture defined successfully.")
print(model.summary())
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is depre
  warnings.warn(
CNN model architecture defined successfully.
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 29)	0	-
embedding (Embedding)	(None, 29, 100)	796,900	input_layer[0][0]
conv1d (Conv1D)	(None, 27, 128)	38,528	embedding[0][0]
conv1d_1 (Conv1D)	(None, 26, 128)	51,328	embedding[0][0]
conv1d_2 (Conv1D)	(None, 25, 128)	64,128	embedding[0][0]
global_max_pooling… (GlobalMaxPooling1…	(None, 128)	0	conv1d[0][0]
global_max_pooling… (GlobalMaxPooling1…	(None, 128)	0	conv1d_1[0][0]
global_max_pooling… (GlobalMaxPooling1…	(None, 128)	0	conv1d_2[0][0]
concatenate (Concatenate)	(None, 384)	0	global_max_pooli… global_max_pooli… global_max_pooli…
dense (Dense)	(None, 128)	49,280	concatenate[0][0]
dense_1 (Dense)	(None, 1)	129	dense[0][0]

Total params: 1,000,293 (3.82 MB)
Trainable params: 203,393 (794.50 KB)

```
input_layer = Input(shape=(max_sequence_length,))

embedding_layer = Embedding(
    input_dim=vocabulary_size,
    output_dim=embedding_dimension,
    weights=[embedding_matrix],
    trainable=False
)(input_layer)

# Define different filter sizes for Conv1D layers
filter_sizes = [3, 4, 5]
conv_blocks = []

for fsize in filter_sizes:
    conv = Conv1D(filters=128, kernel_size=fsize, activation='relu')(embedding_layer)
    pool = GlobalMaxPooling1D()(conv)
    conv_blocks.append(pool)

# Concatenate the outputs of the different convolutional blocks
if len(conv_blocks) > 1:
    merged = concatenate(conv_blocks)
else:
    merged = conv_blocks[0]

dense_layer_1 = Dense(128, activation='relu')(merged)
output_layer = Dense(1, activation='sigmoid')(dense_layer_1)

model = Model(inputs=input_layer, outputs=output_layer)

print("CNN model architecture defined successfully.")
print(model.summary())
```

```
CNN model architecture defined successfully.
Model: "functional_1"
```

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 29)	0	-
embedding_1 (Embedding)	(None, 29, 100)	796,900	input_layer_1[0]…
conv1d_3 (Conv1D)	(None, 27, 128)	38,528	embedding_1[0][0]
conv1d_4 (Conv1D)	(None, 26, 128)	51,328	embedding_1[0][0]
conv1d_5 (Conv1D)	(None, 25, 128)	64,128	embedding_1[0][0]
global_max_pooling… (GlobalMaxPooling1…	(None, 128)	0	conv1d_3[0][0]
global_max_pooling… (GlobalMaxPooling1…	(None, 128)	0	conv1d_4[0][0]
global_max_pooling… (GlobalMaxPooling1…	(None, 128)	0	conv1d_5[0][0]
concatenate_1 (Concatenate)	(None, 384)	0	global_max_pooli… global_max_pooli… global_max_pooli…
dense_2 (Dense)	(None, 128)	49,280	concatenate_1[0]…
dense_3 (Dense)	(None, 1)	129	dense_2[0][0]

```
Total params: 1,000,293 (3.82 MB)
Trainable params: 203,393 (794.50 KB)
Non-trainable params: 796,900 (3.04 MB)
```

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
print("CNN model compiled successfully.")
```

```
CNN model compiled successfully.
```

```
history = model.fit(X_train_padded, y_train, epochs=10, batch_size=32, validation_data=(X_test_padded, y_test))
print("Model training complete. Training history stored in 'history' variable.")
```

```
Epoch 1/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 9s 29ms/step - accuracy: 0.9050 - loss: 0.2660 - val_accuracy: 0.9686 - val_loss: 0.0886
Epoch 2/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 16ms/step - accuracy: 0.9900 - loss: 0.0411 - val_accuracy: 0.9668 - val_loss: 0.0971
Epoch 3/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - accuracy: 0.9986 - loss: 0.0129 - val_accuracy: 0.9749 - val_loss: 0.0776
Epoch 4/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - accuracy: 0.9999 - loss: 0.0032 - val_accuracy: 0.9767 - val_loss: 0.0883
Epoch 5/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - accuracy: 1.0000 - loss: 0.0014 - val_accuracy: 0.9767 - val_loss: 0.0964
Epoch 6/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - accuracy: 1.0000 - loss: 8.0888e-04 - val_accuracy: 0.9776 - val_loss: 0.1013
Epoch 7/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - accuracy: 1.0000 - loss: 4.6940e-04 - val_accuracy: 0.9776 - val_loss: 0.1050
Epoch 8/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 16ms/step - accuracy: 1.0000 - loss: 4.0189e-04 - val_accuracy: 0.9767 - val_loss: 0.1087
Epoch 9/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 3s 18ms/step - accuracy: 1.0000 - loss: 2.6194e-04 - val_accuracy: 0.9767 - val_loss: 0.1160
Epoch 10/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - accuracy: 1.0000 - loss: 2.0625e-04 - val_accuracy: 0.9767 - val_loss: 0.1168
Model training complete. Training history stored in 'history' variable.
```

```
y_pred_proba = model.predict(X_test_padded)
y_pred = (y_pred_proba > 0.5).astype(int)

print("Predictions on the test set generated.")
print(f"Shape of y_pred_proba: {y_pred_proba.shape}")
print(f"First 10 predicted probabilities: {y_pred_proba[:10].flatten()}")
print(f"First 10 predicted labels: {y_pred[:10].flatten()}")
```

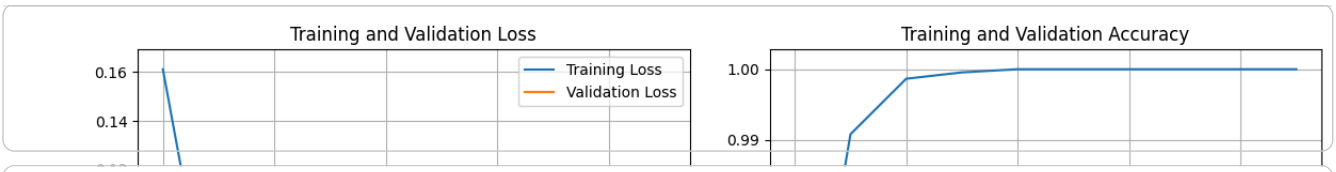
```
35/35 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step
Predictions on the test set generated.
Shape of y_pred_proba: (1115, 1)
First 10 predicted probabilities: [2.0975777e-09 1.7371255e-07 7.5089069e-08 9.9962854e-01 2.1928643e-06
 3.4569121e-06 3.1674578e-08 2.6125283e-04 1.6301366e-06 1.5240778e-04]
First 10 predicted labels: [0 0 0 1 0 0 0 0 0 0]
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f"Accuracy: {accuracy:.4f}")

# Calculate precision
precision = precision_score(y_test, y_pred)
print(f"Precision: {precision:.4f}")

# Calculate recall
recall = recall_score(y_test, y_pred)
print(f"Recall: {recall:.4f}")

# Calculate F1-score
f1 = f1_score(y_test, y_pred)
print(f"F1-score: {f1:.4f}")

# Display classification report for a more detailed breakdown
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.9767
Precision: 0.9556
Recall: 0.8658
F1-score: 0.9085

Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.99      0.99       966
           1       0.96      0.87      0.91       149

    accuracy                           0.98      1115
   macro avg       0.97      0.93      0.95      1115
weighted avg       0.98      0.98      0.98      1115
```

```
import matplotlib.pyplot as plt

# Create a figure with two subplots
plt.figure(figsize=(12, 5))

# Plot training and validation loss
plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Plot training and validation accuracy
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Adjust layout and display plots
plt.tight_layout()
plt.show()
```




```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Generate the confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Plot the confusion matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Ham', 'Spam'], yticklabels=['Ham', 'Spam'])
```